

Predicción de Commodities usando series temporales – Revisión bibliográfica

Universidad del Valle de Guatemala
Ciencia de Datos, sección 10
Castellanos, I; Pérez, M. y Rodríguez R.

Alcance de la revisión

La discusión se fundamenta en los scripts de descarga, consultas SQL, cuadernos y resultados versionados del proyecto. La evidencia ejecutada cubre 2024 y la parte publicada de 2026. El benchmark incluye 71,094,351 registros.

71,094,351 filas en benchmark	12.804 s materialización	1.75× mejor caso conjunto
----------------------------------	-----------------------------	------------------------------

9.1. ¿Qué características de DuckDB resultaron más útiles durante el laboratorio?

La característica más útil fue poder consultar archivos Parquet directamente con SQL, sin importarlos primero a una base de datos. Esto permitió explorar esquemas, contar registros, detectar problemas de calidad y producir agregados sobre decenas de millones de filas desde un proceso local y sin administrar un servidor.

También fueron importantes el procesamiento columnar y vectorizado, la lectura de varios archivos mediante patrones, la proyección y el filtrado sobre Parquet, las funciones analíticas y temporales, y la integración con Python. DuckDB permitió materializar la misma información en una tabla y verificar que las consultas directas y materializadas devolvieran resultados equivalentes. La posibilidad de crear vistas hizo posible definir una capa semántica con año, mes, hora, duración, forma de pago y una bandera de validez, reutilizable por las visualizaciones.

En términos prácticos, DuckDB procesó 71,094,351 filas de 2024 y 2026 en una computadora de ocho CPU lógicos. Esto confirmó que una base embebida puede cubrir análisis de gran volumen sin la complejidad operativa de un servicio de base de datos tradicional.

9.2. ¿Qué ventajas y limitaciones encontró al consultar directamente archivos Parquet?

La principal ventaja fue eliminar la etapa de carga inicial. Los archivos descargados pudieron consultarse inmediatamente y conservarse separados por tipo de taxi y año. Agregar datos consistió en colocar nuevos Parquet en la estructura y ampliar la fuente de la consulta. Como Parquet es columnar, DuckDB puede leer sólo las columnas necesarias y aprovechar filtros y estadísticas del archivo, lo cual reduce entrada/salida. Además, los archivos siguen siendo portables y pueden ser consumidos por otras herramientas.

La consulta directa también favoreció la exploración: no fue necesario mantener una copia materializada durante cada cambio. En el benchmark, incluso sobre 71.1 millones de filas, la consulta de percentiles tardó 6.2094 s sobre Parquet y 6.2060 s sobre la tabla, un empate práctico.

Las limitaciones aparecen cuando se repiten muchos escaneos o cuando los esquemas cambian. En este proyecto fue necesario homologar los nombres tpep/lpep, seleccionar columnas explícitamente y representar `cbd_congestion_fee` como NULL para 2024. Un gran número de archivos pequeños añade sobre costo de apertura y metadatos. Parquet tampoco aporta por sí solo una capa central de reglas, restricciones o control transaccional. Finalmente, la calidad de origen sigue siendo responsabilidad del análisis: se detectaron importes negativos, fechas invertidas, nulos y valores atípicos que el formato no corrige.

9.3. ¿Qué ventajas y limitaciones observó al utilizar tablas materializadas en DuckDB?

La tabla materializada ofreció un esquema común, una sola fuente lógica para el tablero y mejor desempeño en agregaciones repetidas. Para 71,094,351 filas, la mediana de la agregación mensual bajó de 1.2600 s a 0.9041 s (1.39 veces más rápida) y la distribución de pagos bajó de 1.1051 s a 0.6321 s (1.75 veces más rápida). Esto es valioso para un tablero que ejecuta las mismas consultas con frecuencia. La tabla también simplificó la creación de las vistas `viajes_base` y `viajes_analiticos`.

La mejora no fue universal: la consulta de percentiles prácticamente empató y, en algunos escenarios individuales, la tabla fue ligeramente más lenta. Materializar exigió 12.804 s y almacenamiento adicional. Con los tiempos medidos, ese costo se recuperaría después de aproximadamente 15.4 ejecuciones completas de las tres consultas del benchmark. También surge la obligación de refrescar la tabla cuando llegan archivos nuevos y de evitar que quede desactualizada respecto de los Parquet. El archivo DuckDB admite un solo proceso escritor a la vez, por lo que la concurrencia de escritura requiere coordinación.

Resumen del benchmark

Consulta	Parquet	Tabla DuckDB	Resultado
Características/percentiles	6.2094 s	6.2060 s	Empate (1.00×)
Agregación mensual	1.2600 s	0.9041 s	Tabla 1.39×
Pagos y propinas	1.1051 s	0.6321 s	Tabla 1.75×

9.4. ¿Qué ventajas ofrece este flujo frente a cargar todos los datos con Pandas?

El flujo con DuckDB ejecuta filtros, uniones y agregaciones dentro del motor y devuelve a Pandas únicamente resultados resumidos. Cargar todo en DataFrames requeriría descomprimir y representar en memoria cada columna, además de crear objetos intermedios durante las transformaciones. Esa estrategia es poco conveniente para los 71.1 millones de filas del benchmark y aumenta el riesgo de agotar la memoria.

DuckDB aporta ejecución perezosa, paralelismo, operaciones vectorizadas, pushdown de columnas y filtros, y posibilidad de procesar más datos que la memoria disponible. SQL también deja consultas declarativas fáciles de versionar, revisar y reutilizar. Pandas sigue siendo útil al final del flujo para tablas pequeñas y visualizaciones, pero no debe ser la capa principal de escaneo. De hecho, el procesamiento inicial del ejercicio 3 sí convirtió conjuntos completos a DataFrames antes de escribirlos; esa decisión

funcionó con el volumen disponible, pero debería reemplazarse por COPY ... TO desde DuckDB para escalar mejor.

9.5. ¿Qué características permiten incorporar nuevos datos con cambios mínimos?

El sistema separa datos crudos y procesados, organiza los archivos como `data/raw/<tipo>/<año>/`, conserva nombres mensuales predecibles y usa descargadores idempotentes. Cada script omite archivos locales no vacíos, descarga primero a un archivo `.part` y sólo lo renombra al terminar; así una reejecución conserva lo existente y no deja un Parquet incompleto con nombre definitivo.

La homologación genera archivos procesados con columnas comunes y agrega `data_year` y `taxi_type` al consolidar. Las consultas usan UNION ALL, columnas explícitas y filtros por año, mientras que `union_by_name` permite tolerar ciertas diferencias de esquema. Las consultas SQL están separadas de los cuadernos, y la capa de vistas desacopla las visualizaciones del almacenamiento físico.

La experiencia también mostró un límite del diseño actual: varios años están enumerados manualmente en SQL y la materialización vigente sólo incluye 2024 y 2026. Para reducir aún más los cambios, convendría descubrir particiones disponibles desde la ruta, centralizar la lista de años en configuración y generar una vista única sobre todos los Parquet compatibles.

9.6. ¿Qué parte debería automatizarse en un sistema de producción?

Debería automatizarse el flujo completo desde la detección de un archivo nuevo hasta la publicación de indicadores, mediante un orquestador con tareas idempotentes y observables. La automatización no debería limitarse a programar la descarga: debe impedir que datos incompletos o incompatibles lleguen a la capa analítica.

- Detección y descarga incremental, con reintentos, checksum, tamaño esperado y registro de procedencia.
- Validación del esquema y pruebas de calidad: nulos, duplicados, fechas invertidas, importes negativos y rangos plausibles.
- Transformación a Parquet procesado, preservando métricas de imputación y rechazo por archivo.
- Actualización incremental y atómica de la tabla o vistas, seguida de pruebas de igualdad y conteos por partición.
- Ejecución de consultas, actualización del tablero y generación de reportes sólo si las validaciones aprueban.
- Monitoreo con alertas, bitácora, duración, filas procesadas, versión del código y posibilidad de reanudar o revertir una corrida.

9.7. ¿Qué decisiones de diseño fueron importantes para mantener el proyecto reproducible?

La reproducibilidad dependió de versionar código, SQL y cuadernos, pero excluir datos regenerables mediante `.gitignore`. Los scripts reconstruyen la descarga desde la fuente oficial; los artefactos se ubican en carpetas estables; las consultas tienen nombres por ejercicio; y Docker Compose fija un ambiente común. El benchmark registra versión de DuckDB, CPU, cantidad de filas, estrategia, repetición y tiempo, alterna el orden de las estrategias y comprueba igualdad de resultados antes de comparar rendimiento.

También fueron importantes no modificar el descargador original al añadir 2024 y 2025, mantener raw separado de processed, usar compresión ZSTD, filtrar por el año nominal y documentar reglas de limpieza. La comparación temporal debe usar una ventana común cuando un año está incompleto, evitando interpretar cobertura parcial como caída del servicio.

9.8. ¿Qué aprendió sobre datos que no habría sido evidente con conjuntos pequeños?

El volumen cambia qué decisiones son viables. Con 71 millones de filas, una copia completa en memoria, una conversión innecesaria o un escaneo repetido deja de ser un detalle y se convierte en costo de tiempo, memoria y almacenamiento. También se vuelve visible que el rendimiento depende del tipo de consulta: materializar aceleró agregaciones, pero no aportó una mejora apreciable en percentiles. Por ello no existe una estrategia universalmente superior y el costo inicial debe amortizarse con reutilización.

Los problemas raros también adquieren magnitud. En 2026 se encontraron 161,835 importes negativos en Yellow, 1,023 en Green, 15 viajes con drop-off anterior al pickup y millones de valores nulos en campos operativos. Incluso porcentajes pequeños producen muchos casos que pueden sesgar sumas e indicadores. A la vez, una regla aparentemente obvia puede ser incorrecta: más de 7.8 millones de viajes Yellow tenían pasajero nulo o cero y monto positivo, por lo que descartarlos como viajes inexistentes habría eliminado actividad real.

Finalmente, se aprendió que las medias pueden ocultar distribuciones sesgadas: en Green 2026 la distancia media fue 13.39 millas, mientras la mediana fue 2.07. A gran escala son indispensables las medianas, percentiles, banderas de calidad, particiones, validación de esquemas y métricas de cobertura. El reto no es sólo ejecutar una consulta, sino preservar el significado, la trazabilidad y la comparabilidad de los resultados conforme crecen los datos.